

Debreceni Egyetem
Informatikai Kar
Adattudomány és Vizualizáció Tanszék

**Dinamikus objektumelkerülés megerősítéssel
a CARLA szimulátorban**

Témavezető:

Mészáros László
Tanársegéd

Készítette:

Angi Dávid
Adattudomány MSc

Debrecen, 2026

Tartalomjegyzék

1. Bevezetés	3
2. Szimulációs környezet	4
3. Programozás és algoritmus	6
3.1. Megerősítéses tanulás	6
3.2. Python programozási nyelv	8
3.3. Soft Actor-Critic és a Stable-Baselines3	9
4. Szensoradatok előfeldozása, Autóenkóderek	10
4.1. Kamera	10
4.2. LiDAR	10
4.3. Szenzorfuzio	10
5. Jutalom függvény	10
6. Betanult modellek összehasonlítása	10
7. Összegzés	10
Hivatkozások	11

1. Bevezetés

A 21. század egyik ha nem a legnagyobb hatású technológiai átalakulása a közlekedésben zajlik. Az elmúlt másfél évtizedben szemünk láttára kerültek önvezető járművek a kutatólaboratóriumokból a hétköznapi forgalom részévé. Kezdetben vezetéstámogató rendszerek (sávtartó automatika, adaptív tempomat, vészfékasszisztens) kirivó már technológiának számítottak azonban ma már a középkategóriás autókban is alapfelszereltség részét képezik, egyes városokban pedig sofőr nélküli robottaxik szállítják az utasokat. A fejlődés robbanásszerű gyorsulását a szenzorok relatíve olcsóbbá válása, valamint a járművekbe építhető, nagy teljesítményű számítási egységek megjelenése nélkülözhetetlen volt. Ezen felül az adatfeldolgozás és a mélytanulás fejlődése is kulcsszerepet ezen fejlődés történetben mivel, ezek a technológiák teszik lehetővé, a járművek számára környezetük érzékelését.

Az önvezető autók iránti érdeklődést elsősorban a biztonság motiválja. A közúti balesetekben világszerte évente több mint egymillió ember veszti életét [1], és a balesetek döntő többsége emberi hibára – figyelmetlenségre, fáradtságra, rossz helyzetmegítélésre – vezethető vissza.

Egy gép nem fárad el, semmi sem vonja el a figyelmét, és valamint a reakcióideje optimális esetben rövidebb lehet az emberinél. Ez az eset azonban csak akkor valósulhat meg, ha a vezetést végző algoritmus minden helyzetben megbízhatóan, hatékonyan és biztonságosan működik. Nem elég, ha a jármű az esetek túlnyomó részében – egyenes úton, jó időben, kiszámítható hanem nehézkes körülmények között is helyt kell állnia. Ezen oknál fogva avalódi kihívást a ritka, váratlan helyzetek jelentik, például a forgalom dinamikája, egy sávba kerülő akadály vagy egy előzés közben megváltozó forgalmi helyzet. Egyetlen rosszul kezelt helyzet is súlyos balesethez vezethet. (észlelés, predikció, tervezés, szabályozás) bontva oldják meg. Ezek a rendszerek jól értelmezhetők, de minden szabályt és határesetet előre meg kell fogalmazni bennük, ami a valós forgalom sokszínűsége mellett rendkívül nehéz. Alternatívát a megerősítéses tanulás (*reinforcement learning*, RL) kínál, amelyben az ágens nem előre megírt szabályokat követ, hanem a környezettel való interakció során, jutalmak és büntetések alapján maga tanulja meg, hogyan kell cselekednie. Ez a megközelítés különösen alkalmas az olyan döntési feladatokra, mint a sávtartás, az akadályok kikerülése vagy az előzés, ahol a helyes viselkedést nehéz szabályokkal leírni, de könnyű megmondani, hogy egy adott kimenetel jó vagy rossz.

A vezetési feladatot hagyományosan egymásra épülő, alaposan tervezett modulokra bontják. Ezen blokkokat jól megtervezett logikai adatfolyam köti össze amelyet robotikai rendszerek kötnek össze (pl ROS2) [2]. Ilyen modulokra épülő önevezető rendszer például az Autoware Auto [3] amely egy ROS2 alapokra épülő robotikai rendszer önevezető autózáshoz. A rendszer kifinomultsága abban rejlik hogy minden felhasználó saját igényei szerint tudja formálni a rendszert és saját modulokat is adhat hozzá. Az ilyen bonyolult rendszerekkel szemben jelent egy lehetséges alternatívát a megerősítéses tanulás azonban ennek ára van. Az ágens próbálkozások és hibák sorozatán keresztül tanul amelyek költsége igen magas lenne a való világban, mivel

az elfogadható vezetési stratégia alakítása akár több százezer vagy akár több millió lépésen át is történhet. A tanulási folyamat során az ágensnek gyakorolnia kell és felfedeznie saját világát valamint korlátait. Ezt egy valós járművel elvégezni gyakorlatilag kivitelezhetetlen. Egy szenzorokkal felszerelt tesztjármű, a zárt tesztpálya, a biztonsági sofőr és a karbantartás költsége rendkívül magas, egy ütközés pedig nemcsak anyagi kárt okoz, hanem emberi életet is veszélyeztethet. Ráadásul az ágens szeretnénk felkészíteni a váratlan és veszélyes helyzetekre, hogy azokat is megfelelőképp kezelje és elkerülje a baleseteket.

Ezen okoknál fogva a tanítást és a tesztelést szimulált környezetben végeztem. A szimulációra a CARLA-t [4] egy nyílt forráskódú, kifejezetten önvezetés-kutatásra fejlesztett szimulátor, amely valósághű városi és autópálya-környezetet is képes egyaránt szimulálni valamint a fizikai járműmodellt, amelyek elengedhetetlenek a reális valósághű szimulációkhoz.

Egy önvezető rendszer döntései csak annyira lehetnek jók, amennyire pontos a környezetéről alkotott képe, ez a kép pedig kizárólag a gépi látás és érzékelés révén a szenzorokból származik. A szenzor adatok feldolgozása az ágens számára igazi kihívást jelent. A komplex szenzorokból sokszínű és nagy varianciájú adat érkezik. Ilyen elgondolás okán a munkám során különböző szenzor adat előfeldolgozásokat végeztem ezzel optimalizálva a feladatokat az ágens számára.

Munkám során olyan ágens kifejlesztésén szeretnék dolgozni amely képes forgalmi szituációkat(pl előzés) is megfelelőképp biztonságosan kezelni, valamint betartja a KRESZ előírásokat is egyaránt. A fő gondolatot **terapaptommakol2022obstacle** munkája adta, akik a CARLA szimulátorban DQN alapú ágens tanítottak, amely a LiDAR pontfelhő, a pozíció és a sebesség alapján fékezéssel és kitéréssel sikeresen elkerülte az úton parkoló járművet. Ez a munka azonban kizárólag statikus akadályra terjedt ki. Erre építkezve célom a dinamikus objektumok elkerülése, vagyis hogy az ágens a mozgó, lassabban haladó járműveket is biztonságosan, a KRESZ szabályait betartva kerülje ki.

2. Szimulációs környezet

A tanításhoz és a teszteléshez a CARLA (*Car Learning to Act*) szimulátorra [4, 5] esett a választásom. (1. ábra) A CARLA egy nyílt forráskódú UnrealEngine4 alapú szimulációs környezet, amelyet kifejezetten önvezető rendszerek fejlesztésére, tanítására, validálására és tesztelésére hoztak létre. A szimulátor magja C++ nyelven készült, a megjelenítést és a fizikai szimulációt pedig az Unreal Engine 4 (UE4) játékmotor végzi. Ennek köszönhetően a szimulátor valósághű, képet és járműdinamikát nyújt. A program képes a valóságban is gyakran használt önvezető szenzorok – többek között RGB-kamera, mélységkamera, szemantikus szegmentációs kamera, LiDAR, radar, GNSS és IMU – szimulált megfelelőit generálni minden forgalmi szituációban. Továbbá fel van készítve a dinamikus időjárás, napszak realisztikus kezelésére is. A forgalom pedig szabadon állítható és szabályozható. Így az ágens olyan típusú szenzoradatokról tanulhat, mint amilyeneket egy valódi jármű is szolgáltatna.

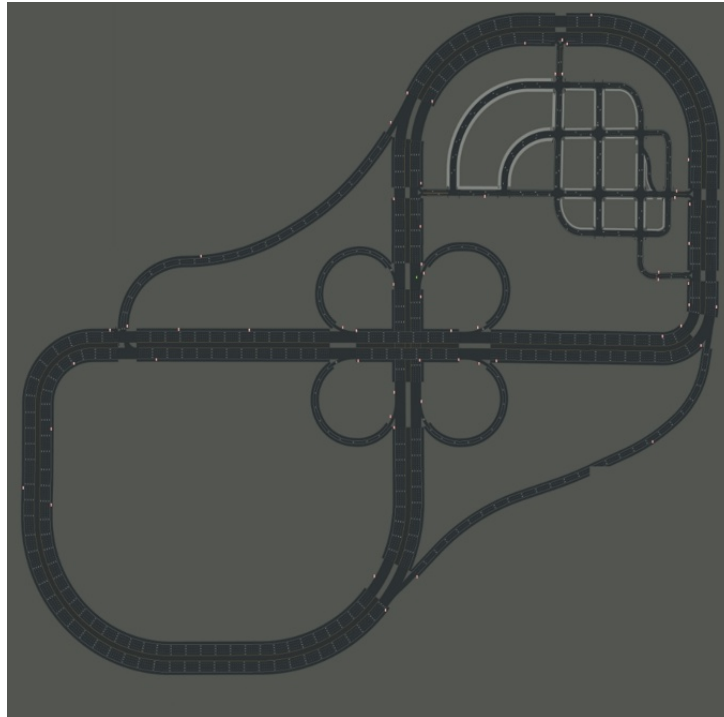


1. ábra. A CARLA szimulátor (forrás: [5])

A szimulátor kliens–szerver architektúrára épül [6], tehát a szerver futtatja magát a szimulációs környezetet ez által ő felel a világ és a járművek fizikájának számításáért, valamint a szenzoradatok előállításáért. Ez a kialakítás ugyanakkor teret enged annak is hogy a munka akár 2 számítógépen is megvalósuljon, például ez akkor indokolt ha nem elegendő egy számítógép grafikus feldolgozási erőforrása arra, hogy a szimuláció és az ágens tanulása is fusson egyaránt. A kliens program, melyet a felhasználó ír, hálózati interfészen (TCP kommunikációs protokoll segítségével alapértelmezetten a 2000-es porton) keresztül kérdezi le a szimuláció állapotát, illetve küld parancsokat a szervernek. Ilyenek lehetnek például a jármű kormány parancsa, gáz adagolása és fékezés is, ugyanakkor a kliens így kapja meg az egyes szimulálni kívánt szenzoradatokat is, a megfigyelni kívánt járműről. A kliensoldali API Pythonban is elérhető, ami nagyban megkönnyíti a fejlesztést. Munkám során a CARLA 0.9.16-os verzióját használtam szinkron módban, rögzített (20 Hz-es) szimulációs lépésközzel. Szinkron módban a szerver csak akkor lép a következő állapotba, amikor a kliens erre utasítja ez nagy segítségemre volt az ágens egyes lépéseinél, így a szenzoradatok és az ágens döntései mindig ugyanahhoz a szimulációs lépéshez tartoztak, valamint a tanítás is reprodukálható marad.

A CARLA-ban adott a lehetőség útvonaltervezésre is. A pálya úthálózata útpontokból (*waypoint*) álló topológiai gráfként érhető el, amelyen a szimulátorhoz tartozó *GlobalRoutePlanner* segítségével két tetszőleges pont között játszani könnyedséggel tervezhető [6] bármilyen kívánt útvonal. Ezt kihasználva minden epizód elején véletlenszerűen választott kezdő- és célpont között új, dinamikus útvonalat tervezek, amelyen az ágensnek végig kell haladnia ezzel kiküszöbölve azt, hogy esetlegesen rátanuljon az adott útvonalra, így segítem az általánosítási képességét.

A szimulátor alapértelmezetten tartalmaz több előre elkészített pályát (*Town*), amelyek varianciája az átlagos önevezető feladatok többségét kielégíti. Munkám során ezek közül a



2. ábra. A Town04 pálya felülnézeti térképe (forrás: [6])

Town04-et választottam (2. ábra). Választásom erre a pályára esett mivel az itt kialakított kisváros tökéletesen megfelel az elvárásaimnak. Alakja „8-as” alakú, végtelenített autópályával [6] megfelelő céloim eléréséhez. A pálya egyszerre tartalmaz városi utcákat és többsávos autópályát, fel- és lehajtókkal, valamint lóhere alakú csomóponttal. Amint említettem célom az autópályás előzés megtanítása, ez a pálya a legalkalmasabb. A többsávos autópályán az ágens a lassabban haladó járművet a bal oldali sávba kihúzódva szabályosan meg tudja előzni, a körbeérő autópálya pedig hosszú, megszakítás nélküli vezetési szakaszokat tesz lehetővé.

3. Programozás és algoritmus

A következőkben munkám elméleti és technikai hátterét írom le röviden. Elsőként a megerősítéses tanulás alapjai majd, ezt követően a fejlesztéshez használt programozási nyelvet és könyvtárait, végül a Soft Actor-Critic algoritmust és a Stable-Baselines3 keretrendszert. Mindegyiknél kitérve arra, miért esett az adott technológiára a választásom.

3.1. Megerősítéses tanulás

A gépi tanulás módszereit hagyományosan három nagy csoportba soroljuk. A *felügyelt tanulás* során a modell tisztában az elvárt kimenet valamint a bementi adatokkal egyaránt így tanul, vagyis minden tanítópéldához ismert a helyes válasz. Ide tartoznak a klasszikus osztályozási és regressziós feladatok, például képeken látható objektumok felismerése, de maga a vezetés is

megtanulható így. A *viselkedésklónozás (behavior cloning)* során a modell egy emberi sofőr rögzített vezetéséből, a szenzoradatok és a hozzájuk tartozó kormány valamint gázparancsok párjaiból tanulja meg utánózni a sofőrt. Ezt az elvet már az 1980-as évek végén alkalmazta az ALVINN rendszer [7]. A módszer korlátja, hogy a modell legfeljebb olyan jó lehet, mint a tanítóadat. Érttem ez alatt, hogy a modell nagy varianciájú adaton kell tanuljon különben a váratlan helyzeteket nem tudja megfelelőképp kezelni. Általánosítási képessége kevésbé erős mint a többi gépitanyúási megoldásnak. Következő módszer *felügyelet nélküli tanulás* ahol nincsenek címkék, a tanítás célja az adatok rejtett szerkezetének, például csoportjainak vagy tömör reprezentációjának feltárása. Tipikus példája a klaszterezés (például a k -közép módszer), amely a hasonló adatpontokat csoportokba rendezi.

A *megegyesítéses tanulás (reinforcement learning, RL)* ezektől alapvetően lényegesen eltér. A tanuló ágensnek senki sem mondja meg, mi lett volna a helyes döntés egy adott helyzetben vagy mi lett volna az elvárt viselkedés, csupán egy számszerű visszajelzést, *jutalmat* kap viselkedéséről [8]. A tanulási folyamatnak két szereplője van egyik az *agens*, a döntések meghozataláért felelős a *környezet*, amelyben az ágens mozog a folyamat során. Ezen páros kölcsönhatása egymással diszkrét időlépésekben történik: az ágens minden lépésben megfigyeli a környezet aktuális állapotát, ez alapján végrehajt egy *akciót*, amelynek hatására a környezet új állapotba kerül, az ágens pedig jutalmat kap előző döntései alapján. A feladatot Markov-döntési folyamatként (*Markov Decision Process*, MDP) írhatjuk fel [8], amiben az állapotok, az akciók, és az állapotátmenet-valószínűségek valamint a jutalomfüggvény határozza meg. A fentiek alapján a modell feltevése a *Markov-tulajdonság*. A következő állapot csak a jelenlegi állapottól és akciótól függ. Mivel az ágens a világot szenzorain keresztül érzékeli, ezt tekintjük állapotnak.

Az ágens viselkedését a *stratégia (policy)* írja le, célja minden tanulási epizódban a *hozam (jutalom)* várható értékének maximalizálása:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \quad (1)$$

ahol r_{t+1} a t -edik lépés utáni jutalom, a $\gamma \in [0, 1)$ diszkontfaktor pedig a jövőbeli jutalmak súlyát szabályozza. A tanulás *epizódokra* tagolódik, amelyek nálam ütközéssel vagy az útvonal teljesítésével érnek véget.

Esetemben nagy valamint, folytonos állapottérben (kamerakép, LiDAR) az értékfüggvényt és a stratégiát mély neurális hálóval közelítem (*deep reinforcement learning*). Ezen típusu algoritmusok között az áttörést a DQN hozta [9], amely *visszajátszási pufferrel (replay buffer)* és *célhálóval (target network)* stabilizálta a tanulást.

Az algoritmusválasztás szempontjából a fő különbségek [8]:

- **Modellalapú vs. modellmentes:** a CARLA fizikájának megtanulása nehéz lenne, ezért modellmentes módszert választottam.

- **Értékalapú, stratégiaalapú, aktor–kritikus:** utóbbiban az *aktor* a stratégia, a *kritikus* pedig értékeli a döntéseit.
- **On-policy vs. off-policy:** az on-policy módszerek (pl. PPO [10]) eldobják a régi tapasztalatot, az off-policy módszerek (DDPG [11], TD3 [12], SAC [13]) újrahasznosítják, így kevesebb interakcióból tanulnak.
- **Diszkrét vs. folytonos akcióter:** a DQN csak véges akciók közül választhat.

3.2. Python programozási nyelv

A kísérletem implementálására a Python programozási nyelvet választottam ami, magas szintű, interpretált, dinamikusan típusos valamint általános célú. Megálmodója Guido van Rossum a nyelv első változata 1991-ben jelentette meg [14]. Tervezési filozófiájának középpontjában az olvashatóság áll. Ezen elvre kiváló példa a kódblokkokat behúzás jelöli, a szintaxis tömör és közel áll a természetes nyelvhez, így a programok gyorsan megírhatók és könnyen érthetők. Ezen tulajdonságainak köszönheti, hogy ma a világ egyik legszélesebb körben használt programozási nyelve, a gépi tanulás és az adattudomány területén pedig gyakorlatilag szabványnak számít.

Elsőre ellentmondásosnak tűnik, hogy egy rendkívül számításigényes gépi tanulási problémát egy viszonylag lassú, nyelvben történő megvalósítás követ, de a magyarázat igen logikus. A Python ezen a területen „ragasztónyelvként” működik. Értem ez alatt, hogy a nagy számítási igényű műveleteket, a mátrixszorzás, konvolúció vagy a gradiensszámítás, C, C++ és CUDA nyelven írt, optimalizált könyvtárak végzik a processzoron vagy a grafikus kártyán. A Python pedig ezen optimalizált könyvtárak kényelmes, magas szintű használatát biztosítja. Az interpreter által okozott műveletek többletideje elhanyagolható, köszönhetően ezen könyvtárak natív implementációjának. A fejlesztő így egy könnyen írható nyelv kényelmét élvezzi, a számítások mégis natív sebességgel futnak.

A neurális hálókat PyTorch [15] keretrendszerben valósítottam meg. Ez GPU-gyorsított tenzorműveleteket és automatikus differenciálást biztosít, a számítási gráfot pedig futás közben építi fel, ami megkönnyíti a fejlesztést és a hibakeresést. A választást az is indokolta, hogy a Stable-Baselines3 is PyTorch-ra épül, így a későbbi neurális hálók és az ágens hálói egy keretrendszerben, közös GPU-n futhatnak.

A Python mellett szólt végül de nem utolsó sorban, hogy a CARLA kliensoldali API-ja is elérhető Pythonban (2. fejezet) ámbár a kliens maga eredetileg C++ nyelven íródott. Így egyetlen nyelven valósulhatott meg a szimulátor vezérlése, a szenzoradatok feldolgozása, a neurális hálók tanítása és maga a megerősítéses tanulás is.

3.3. Soft Actor-Critic és a Stable-Baselines3

A Soft Actor-Critic (SAC) off-policy aktor–kritikus algoritmus, amely folytonos akciótérben sztochasztikus stratégiát tanul. A várható hozam mellett a stratégia entrópiáját is maximalizálja:

$$J(\pi) = \sum_t \mathbb{E} \left[r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t)) \right], \quad (2)$$

ahol $\mathcal{H}$ a stratégia entrópiája, az α hőmérséklet pedig a jutalom és az entrópia súlyozása. Így a felfedezés magából a célfüggvényből következik: az ágens nem ragad le korán egy rossz megoldásnál, és ha több, közel azonos értékű viselkedés létezik – például most előzzön vagy kicsit később –, mindegyiket megtartja, ami robusztusabbá teszi a tanult stratégiát.

Az aktor minden állapothoz egy valószínűségi eloszlást rendel az akciók felett, és ebből mintavételezzük a végrehajtott akciót. A két kritikus függetlenül becsli a Q-függvényt, és a tanítás a kisebbik becslést használja, ami csökkenti a túlbecslést. A célértékeket lassan frissülő célhálók számítják, a tanítás a visszajátszási pufferből vett kötegeken történik, az α értékét pedig az algoritmus automatikusan hangolja: a tanítás elején több felfedezést enged, később a megtanult viselkedést részesíti előnyben. A SAC stabilabban tanul a korábbi off-policy módszerekénél, és Chen, Yuan és Tomizuka [16] CARLA-beli városi vezetési összehasonlításában is a legjobb eredményt érte el.

Az algoritmust nem magam implementáltam, hanem a Stable-Baselines3 (SB3) [17] könyvtárat használtam. Az RL implementációk különösen hibára hajlamosak, mert egy elrontott részlet nem okoz hibaüzenetet, csak csendben rontja a tanulást. Az SB3 tesztelt, PyTorch-alapú megvalósításokat ad egységes felülettel, így a munkám a környezetre, a megfigyelésekre és a jutalomra összpontosulhatott.

Az SB3 a Gymnasium [18] szabványnak megfelelő környezetekkel működik. Egy ilyen környezet leírja a megfigyelések és akciók terét, a reset függvénnyel új epizódot indít, a step függvénnyel pedig végrehajtja az akciót, és visszaadja az új megfigyelést, a jutalmat és az epizód végét jelző értékeket. A szabvány megkülönbözteti a valódi végállapotot (például ütközés) az időkorlát miatti megszakítástól, mert utóbbi után a vezetés folytatható lenne, és ezt a tanulásnak is figyelembe kell vennie.

A keretrendszerhez saját magam valósítottam meg:

- a CARLA-környezetet a Gymnasium interfész szerint (jármű, szenzorok, útvonalak, forgalom, epizódkezelés);
- a megfigyelések előállítását a nyers szenzoradatokból (4. fejezet);
- a jutalomfüggvényt és az epizódvég feltételeit (5. fejezet);

- a kamera, a LiDAR, a járműadatok és az útvonal feldolgozását és összefűzését végző jellemzőkinyerő hálót;
- a tanítás fontosabb mutatóit TensorBoardba naplózó callback-függvényeket.

4. Szenzoradatok előfeldozása, Autóenkóderek

4.1. Kamera

4.2. LiDAR

4.3. Szenzorfuzio

5. Jutalom függvény

6. Betanult modellek összehasonlítása

7. Összegzés

Hivatkozások

- [1] World Health Organization. *Global Status Report on Road Safety 2023*. Geneva: World Health Organization, 2023. ISBN: 978-92-4-008651-7. URL: <https://www.who.int/publications/i/item/9789240086517> (elérés dátuma 2026. 09. 29.).
- [2] Steven Macenski és tsai. „Robot Operating System 2: Design, Architecture, and Uses in the Wild”. *Science Robotics* 7.66 (2022), eabm6074. DOI: 10.1126/scirobotics.abm6074.
- [3] Shinpei Kato és tsai. „Autoware on Board: Enabling Autonomous Vehicles with Embedded Systems”. *2018 ACM/IEEE 9th International Conference on Cyber-Physical Systems (ICCPS)*. 2018, 287–296. old. DOI: 10.1109/ICCPS.2018.00035.
- [4] Alexey Dosovitskiy és tsai. „CARLA: An Open Urban Driving Simulator”. *Proceedings of the 1st Annual Conference on Robot Learning*. 2017, 1–16. old.
- [5] CARLA Team. *CARLA: Open-source Simulator for Autonomous Driving Research*. URL: <https://carla.org> (elérés dátuma 2026. 10. 01.).
- [6] CARLA Team. *CARLA Simulator Documentation (0.9.16)*. URL: <https://carla.readthedocs.io/en/0.9.16/> (elérés dátuma 2026. 10. 01.).
- [7] Dean A. Pomerleau. „ALVINN: An Autonomous Land Vehicle in a Neural Network”. *Advances in Neural Information Processing Systems 1 (NIPS)*. 1989, 305–313. old.
- [8] Richard S. Sutton és Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2. kiad. Cambridge, MA: MIT Press, 2018. ISBN: 978-0-262-03924-6.
- [9] Volodymyr Mnih és tsai. „Human-level Control through Deep Reinforcement Learning”. *Nature* 518.7540 (2015), 529–533. old. DOI: 10.1038/nature14236.
- [10] John Schulman és tsai. „Proximal Policy Optimization Algorithms”. *arXiv preprint arXiv:1707.06347* (2017).
- [11] Timothy P. Lillicrap és tsai. „Continuous Control with Deep Reinforcement Learning”. *4th International Conference on Learning Representations (ICLR)*. 2016.
- [12] Scott Fujimoto, Herke van Hoof és David Meger. „Addressing Function Approximation Error in Actor-Critic Methods”. *Proceedings of the 35th International Conference on Machine Learning (ICML)*. 80. köt. Proceedings of Machine Learning Research. 2018, 1587–1596. old.
- [13] Tuomas Haarnoja és tsai. „Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor”. *Proceedings of the 35th International Conference on Machine Learning (ICML)*. 80. köt. Proceedings of Machine Learning Research. 2018, 1861–1870. old.

- [14] Python Software Foundation. *The Python Programming Language*. URL: <https://www.python.org> (elérés dátuma 2026. 10. 02.).
- [15] Adam Paszke és tsai. „PyTorch: An Imperative Style, High-Performance Deep Learning Library”. *Advances in Neural Information Processing Systems 32 (NeurIPS)*. 2019, 8024–8035. old.
- [16] Jianyu Chen, Bodi Yuan és Masayoshi Tomizuka. „Model-free Deep Reinforcement Learning for Urban Autonomous Driving”. *2019 IEEE Intelligent Transportation Systems Conference (ITSC)*. 2019, 2765–2771. old. doi: 10.1109/ITSC.2019.8917306.
- [17] Antonin Raffin és tsai. „Stable-Baselines3: Reliable Reinforcement Learning Implementations”. *Journal of Machine Learning Research* 22.268 (2021), 1–8. old.
- [18] Mark Towers és tsai. „Gymnasium: A Standard Interface for Reinforcement Learning Environments”. *arXiv preprint arXiv:2407.17032* (2024).